

Conception d'un filtre pour la réduction des bruits parasites lors d'un appel téléphonique

BUZEL NOËL Taïнна

N° SCEI 20519

June 10, 2026

Introduction

Contexte

En Guyane française, des bruitages nocturnes d'insectes et de grenouilles perturbent la qualité des appels téléphoniques. Pour y remédier, les filtres antiparasites traitent en continu chaque échantillon audio pour atténuer ces bruits indésirables, améliorant ainsi la clarté des communications malgré les contraintes environnementales.

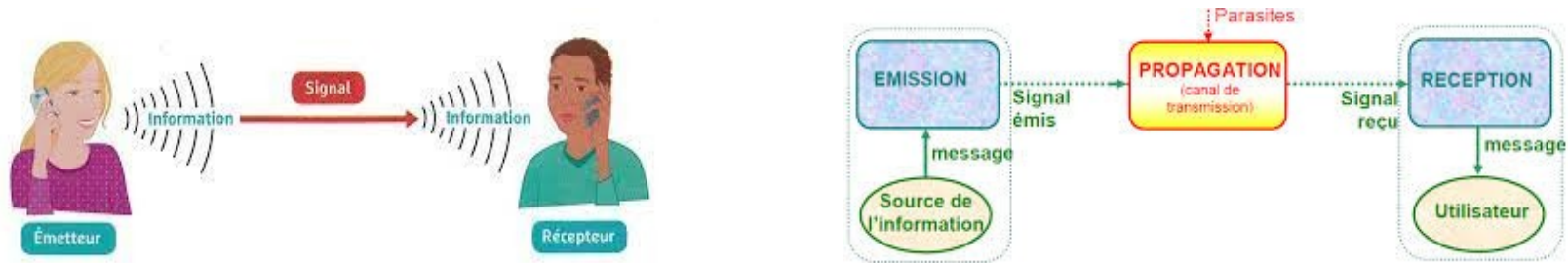


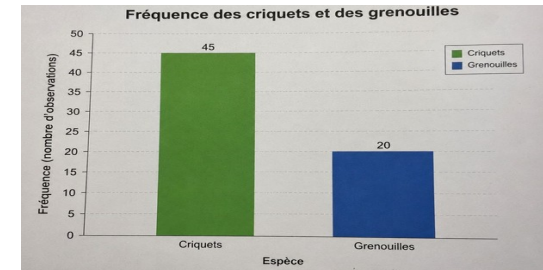
Figure 1 : exemples de schéma de propagation du signal lors d'un appel

PROBLEMATIQUE

Face aux perturbations causées par les insectes nocturnes en Guyane, comment peut-on concevoir un système de filtrage audio pour améliorer la qualité des appels téléphoniques sans modifier la voix humaine ?

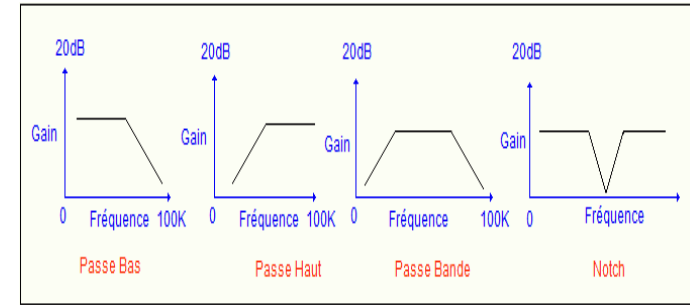
OBJECTIFS SCIENTIFIQUES

- Identifier les fréquences caractéristiques des insectes.
- Comparer plusieurs stratégies de filtrage.
- Concevoir un prototype analogique et numérique.
- Évaluer quantitativement les performances.



OBJECTIFS SCIENTIFIQUES

- Identifier les fréquences caractéristiques des insectes.
- Comparer plusieurs stratégies de filtrage.
- Concevoir un prototype analogique et numérique.
- Évaluer quantitativement les performances.



OBJECTIFS SCIENTIFIQUES

- Identifier les fréquences caractéristiques des insectes.
- Comparer plusieurs stratégies de filtrage.
- Concevoir un prototype analogique et numérique.
- Évaluer quantitativement les performances.



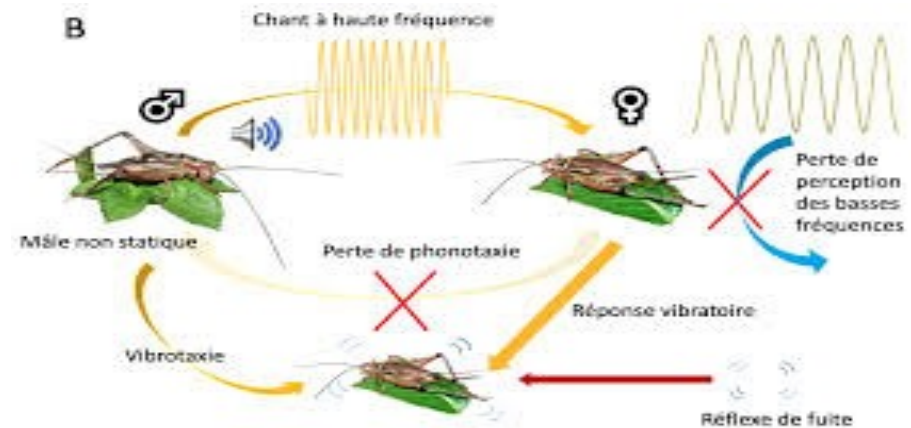
OBJECTIFS SCIENTIFIQUES

- Identifier les fréquences caractéristiques des insectes.
- Comparer plusieurs stratégies de filtrage.
- Concevoir un prototype analogique et numérique.
- Évaluer quantitativement les performances.



ANALYSE PHYSIQUE DU PROBLEME

- La voix humaine est principalement comprise entre 300 Hz et 3400 Hz.
- Les insectes produisent des signaux quasi périodiques.
- L'analyse fréquentielle permet de séparer parole et bruit.

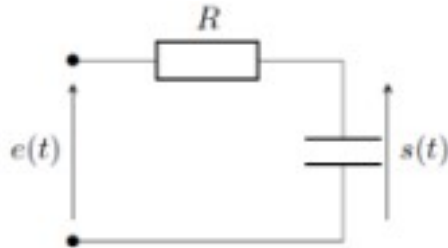


SOLUTIONS

→ L'usage de filtres sont nécessaires afin de pouvoir atténuer le signal reçu après la sommation des deuc signaux.

Pour cela, deux filtres ont été étudiés afin de pouvoir correctement filtrer le son de la voix :

Filtre Passe Bas



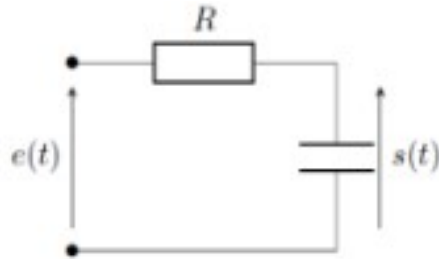
$$H(jx) = H_0 \times \frac{1}{1+jx}$$

SOLUTIONS

→ L'usage de filtres sont nécessaires afin de pouvoir atténuer le signal reçu après la sommation des deux signaux.

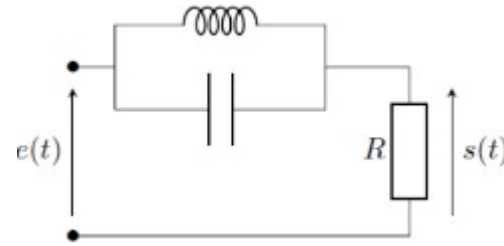
Pour cela, deux filtres ont été étudiés afin de pouvoir correctement filtrer le son de la voix :

Filtre Passe Bas



$$H(jx) = H_0 \times \frac{1}{1+jx}$$

Filtre Coupe Bande



$$H(jx) = H_0 \times \frac{1-x^2}{1-x^2+j\frac{x}{Q}}$$

REFLEXION 1

DEMARCHE EXPERIMENTALE

- Enregistrer la voix et les bruits parasites.
- Calculer le spectre fréquentiel (FFT).
- Identifier les fréquences dominantes à supprimer.

MODELISATION SOUS PYTHON

- Simulation d'un signal voix + bruit.
- Étude de l'effet des paramètres du filtre.
- Comparaison avant/après traitement.

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.io.wavfile as sci_wav

fe, signal_avec_bruit=sci_wav.read("/home/tainnouch/TIPE/perturbations.wav")
son = signal_avec_bruit[:,0] + signal_avec_bruit[:,1]
print("La fréquence d'échantillonnage est de:" ,fe, "Les données du signal sonore sont:", son)
Te=1/fe
n=len(son)

Fourier=np.fft.rfft(son,n)
freq=np.fft.rfftfreq(n,Te)
Fourier=np.abs(Fourier)
plt.xlim(0,1000)
plt.plot(freq,Fourier)
plt.show()

##### Filtre passe bas : méthode 1

def filtrage_passe_bas(signal_non_filtré, fc):
    Ttotal = len(son)*Te
    Ttest = np.linspace(0,Ttotal,len(son))
    T= Ttest[:-1]

    signal_filtré=np.zeros(len(signal_non_filtré)-1)
    omega0=2*np.pi*fc

    for i in range(len(signal_non_filtré)-2):
        signal_filtré[i+1]=(1-omega0*Te)*signal_filtré[i] + omega0*Te*signal_non_filtré[i]
    print(len(Ttest), len(signal_non_filtré), len(signal_filtré))
```

REFLEXION 2

```
Fourier=np.fft.rfft(signal_filtre,n)
freq=np.fft.rfftfreq(n,Te)
Fourier=np.abs(Fourier)
plt.xlim(0,1000)
plt.plot(freq,Fourier,color='orange')
plt.show()
```

```
signal_final= signal_filtre.astype('int16')
sci_wav.write("signalfiltré.wav", fe, signal_final)
signal_non_filtre=np.abs(signal_non_filtre)
plt.plot(Ttest,signal_non_filtre)
signal_filtre=np.abs(signal_filtre)
plt.plot(T,signal_filtre,color='red')
plt.show()
```

#signal filtré : voix ok mais toujours du bruit parasite

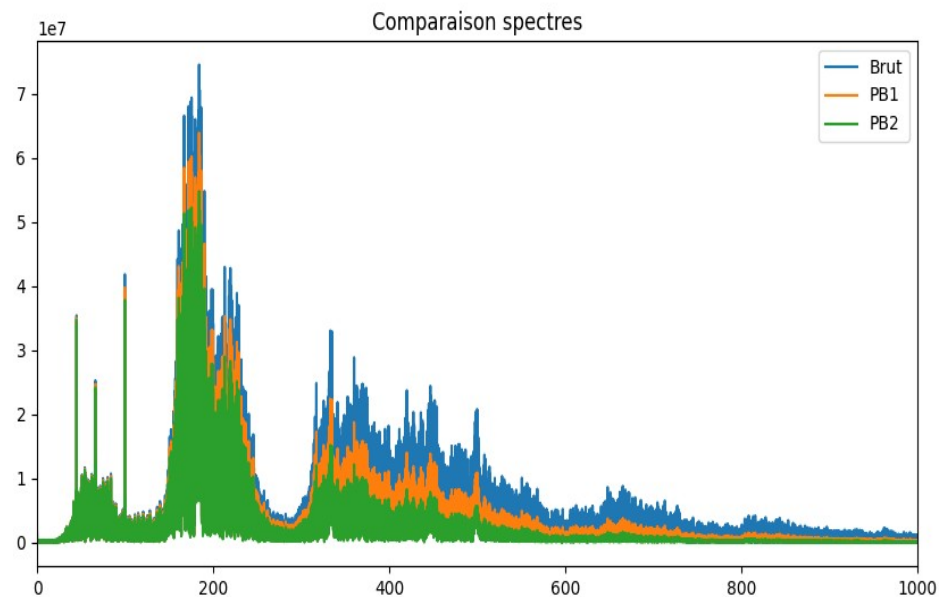
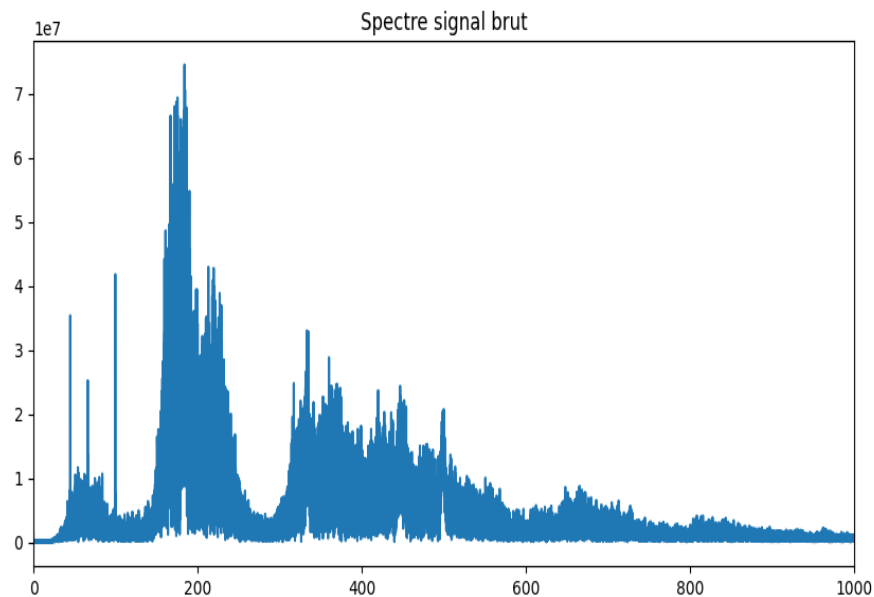
```
signal_filtre2=np.zeros(len(signal_non_filtre)-1)
for i in range(len(signal_non_filtre)-2):
    signal_filtre2[i+1]=(1-omega0*Te)*signal_filtre2[i] + omega0*Te*signal_filtre[i]
```

```
Fourier=np.fft.rfft(signal_filtre2,n)
freq=np.fft.rfftfreq(n,Te)
Fourier=np.abs(Fourier)
plt.xlim(0,1000)
plt.plot(freq,Fourier,color='green')
plt.show()
```

```
signal_non_filtre=np.abs(signal_non_filtre)
plt.plot(Ttest,signal_non_filtre)
signal_filtre2=np.abs(signal_filtre2)
plt.plot(T,signal_filtre2,color='black')
plt.show()
```

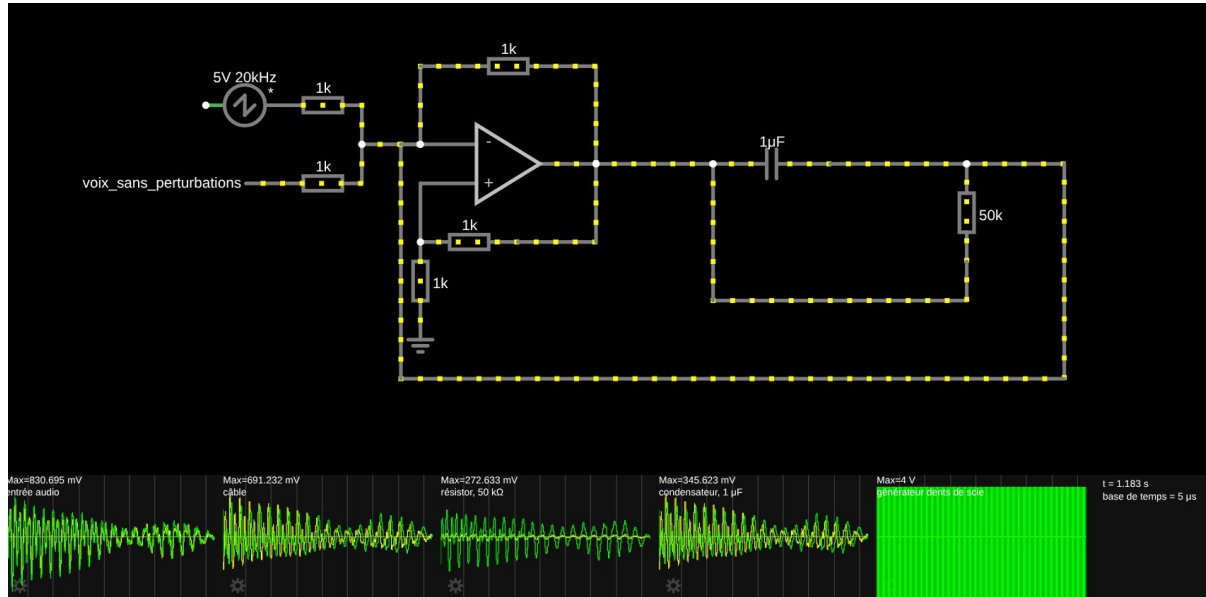
```
signal_final2= signal_filtre2.astype('int16')
sci_wav.write("signalfiltré2.wav", fe, signal_final2)
```

GRAPHES OBTENUS



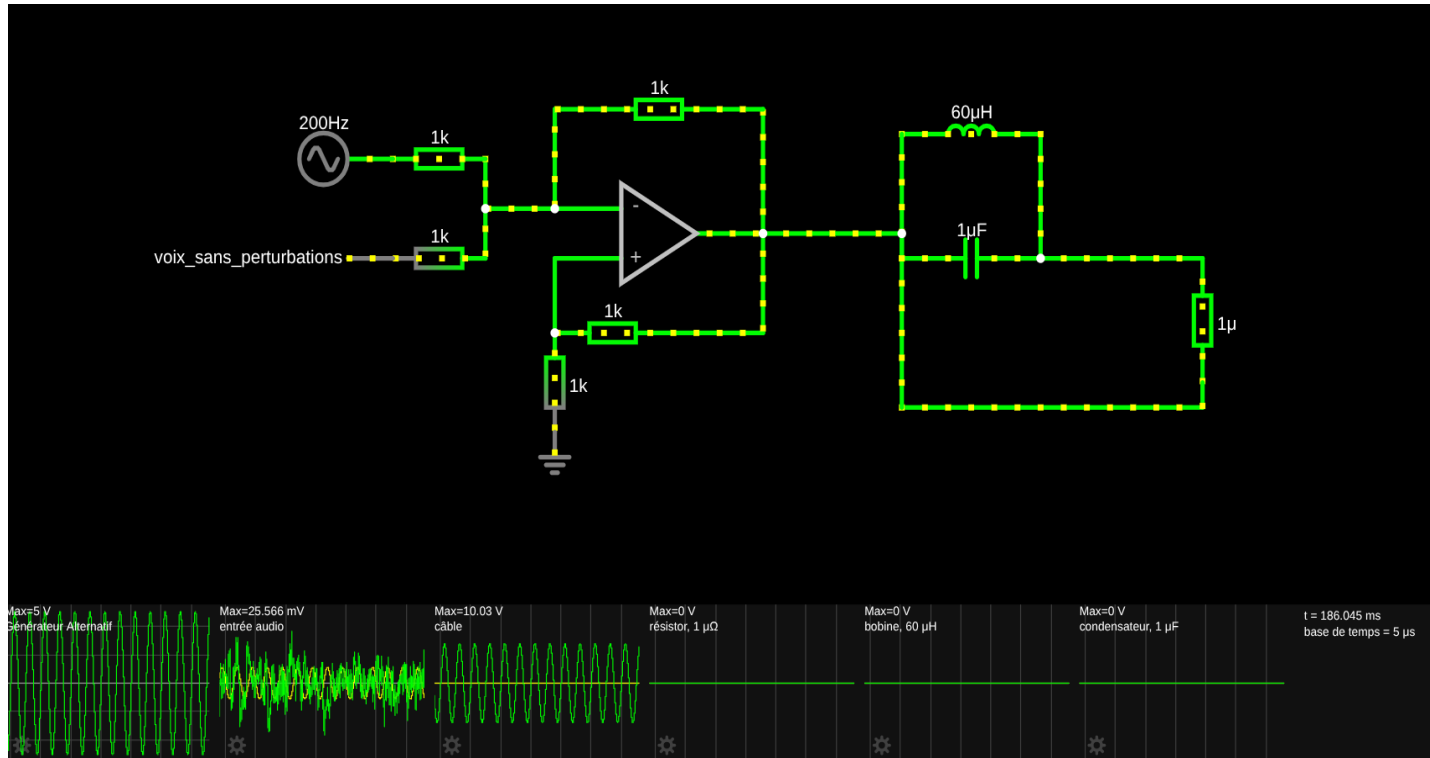
CONCEPTION DU FILTRE ANALOGIQUE 1

- Utilisation d'amplificateurs opérationnels.
- Sommation des signaux.
- Dimensionnement des composants à partir de la fréquence de coupure.



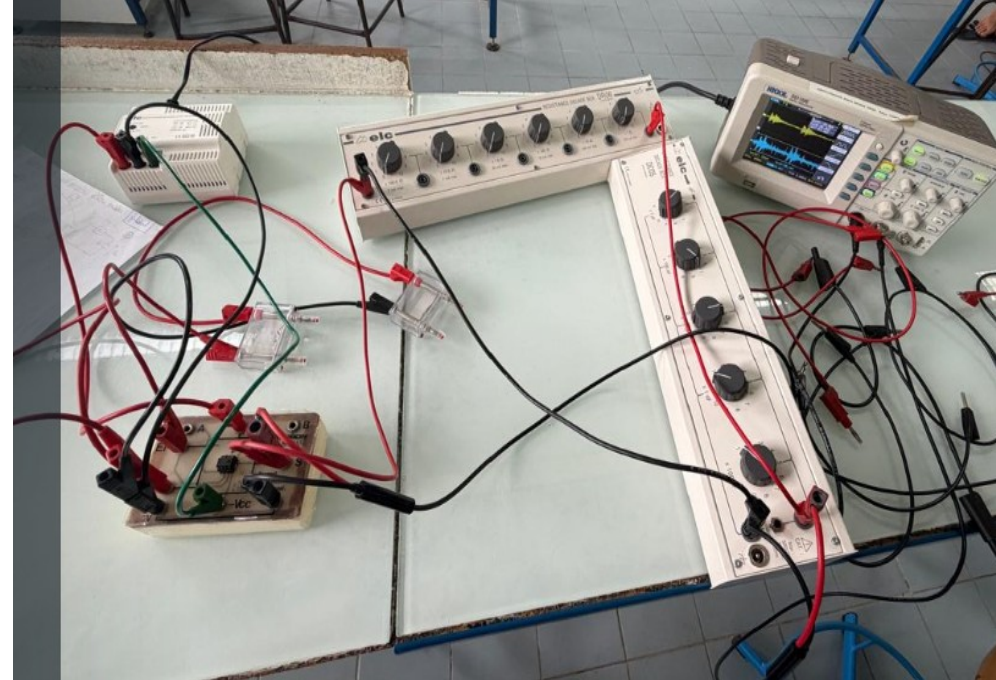
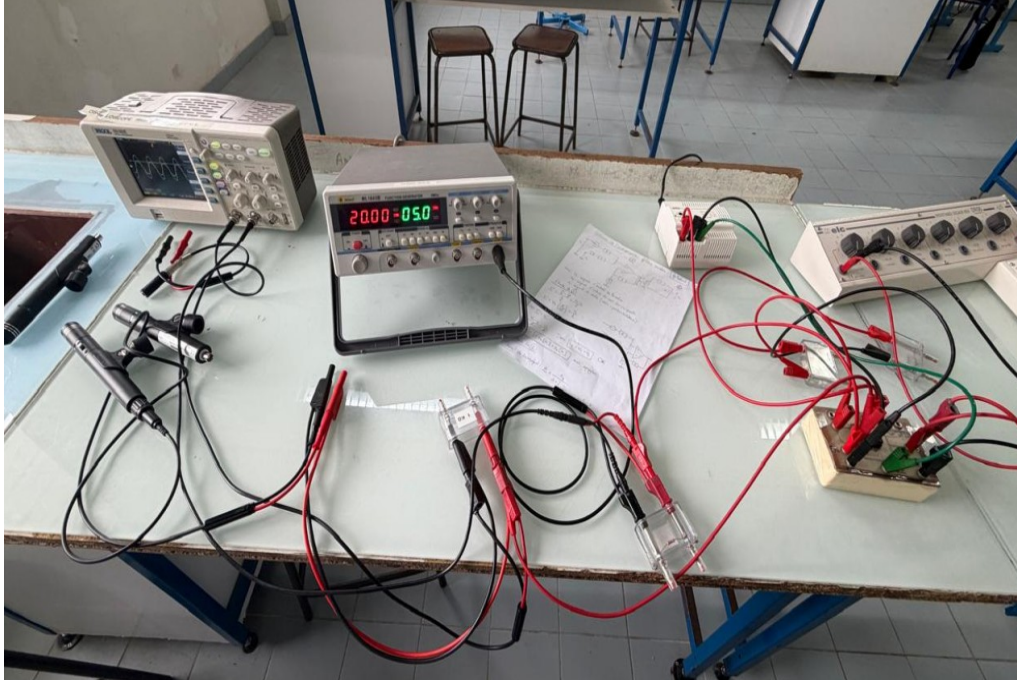
Expérience sur le
logiciel FALSTAD
(PASSE BAS)

CONCEPTION DU FILTRE ANALOGIQUE 2



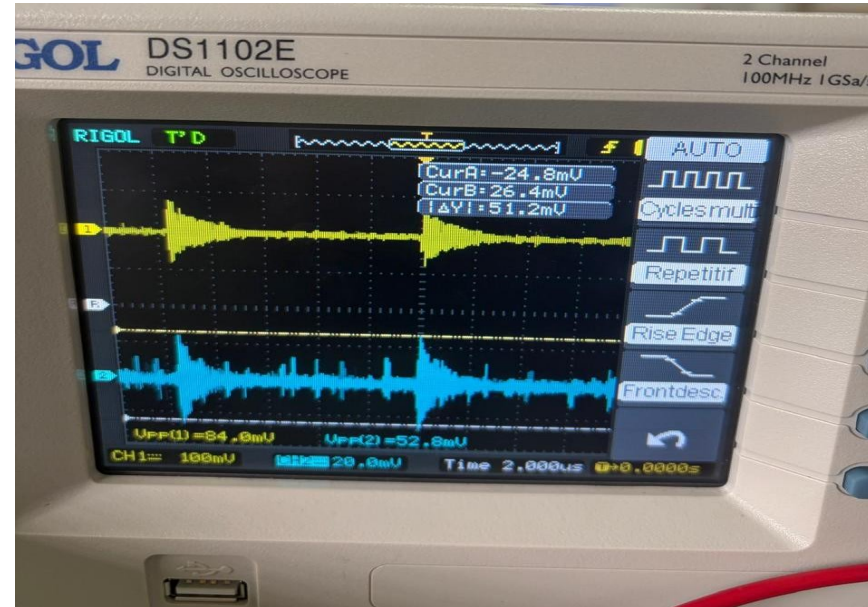
Expérience sur le
logiciel FALSTAD
(COUPE
BANDE)

CONCEPTION DU FILTRE ANALOGIQUE REEL



Critères d'évaluation

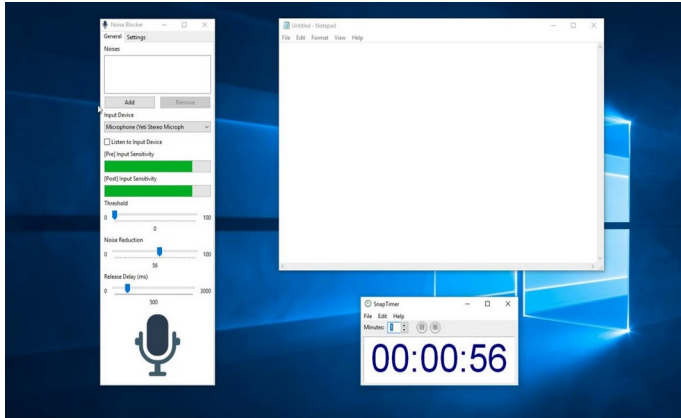
- Atténuation du bruit (dB).
- Conservation de la voix.
- Rapport signal/bruit (SNR).
- Écoute comparative.



Atténuation
du signal de
sortie en réel

Ouverture technologique

- Implémentation sur Arduino.
- Traitement en temps réel.
- Application aux communications mobiles.



Application :
Noise blocker



Carte
Arduino

MERCI POUR VOTRE ATTENTION